Wreath-product circuit value classification by universal leaf languages and the center quotient

Laurent Bartholdi, Michael Figelius
Markus Lohrey, Armin Weiß

doi:10.4230/LIPIcs.CCC.2020.29; CC BY 3.0 Unported

Editorial scope. Independent formal Theorem6 universal leaf-language characterization, full AppendixC support chain grouped once. Does not claim full PSPACE corollary or uniform SENS padding proof.

Allowed standard prerequisites (editorial).

- Straight-line programs, leaf languages, standard super-decreasing subset-sum SLP encoding. Full local conventions and original Lemmas28–33 retained. Standard cited results[29,35,38] are imported with exact stated conditions; new circuit-to-subsetsum and all wreath/center witness constructions fully retained. Full51 original references.

Wreath-product circuit-value classification (Theorem 6)

Let G be a f.g. non-trivial group with center $Z = Z(G)$.

- $\mathrm{CVP}(G \wr \mathbb{Z})$ belongs to $\forall\mathrm{LEAF}(\mathrm{WP}(G))$.
- $\mathrm{CVP}(G \wr \mathbb{Z})$ is hard for the class $\forall\mathrm{LEAF}(\mathrm{WP}(G/Z))$.

In particular, if $Z = 1$, then $\mathrm{CVP}(G \wr \mathbb{Z})$ is complete for $\forall\mathrm{LEAF}(\mathrm{WP}(G))$.

Wreath-product conventions (Section 2)

A fundamental group construction that we shall use is the *wreath product*: given groups G and H acting on the right on sets X and Y respectively, their *wreath product* $G \wr H$ is a group acting on $X \times Y$. We start with the restricted direct product $G^{(Y)}$ (the base group) of all mappings $f : Y \to G$ having finite support $\mathrm{supp}(f) = \{y \mid f(y) \neq 1\}$ with the operation of pointwise multiplication. The group H has a natural left action on $G^{(Y)}$: for $f \in G^{(Y)}$ and $h \in H$, we define ${}^h f \in G^{(Y)}$ by $({}^h f)(y) = f(y^h)$. The corresponding semidirect product $G^{(Y)} \rtimes H$ is the *wreath product* $G \wr H$. In other words:

- Elements of $G \wr H$ are pairs $(f, h) \in G^{(Y)} \times H$ and we simply write fh for this pair.

- The multiplication in $G \wr H$ is defined as follows: Let $f_1h_1, f_2h_2 \in G \wr H$. Then $f_1h_1f_2h_2 = f_1{}^{h_1}f_2h_1h_2$, where the product $f_1{}^{h_1}f_2 \colon y \mapsto f_1(y)f_2(y^{h_1})$ is the pointwise product.

The wreath product $G \wr H$ acts on $X \times Y$ by $(x, y)^{fh} = (x^{f(y)}, y^h)$. The wreath product defined above is also called the *(restricted) permutational wreath product*. There is also the variant where $G = X$ and $H = Y$ and both groups act on themselves by right-multiplication, which is called the *(restricted) regular wreath product* (or *standard wreath product*). A subtle point is that the permutational wreath product is an associative operation whereas the regular wreath product is in general not. The term "restricted" refers to the fact that the base group is $G^{(Y)}$, i.e., only finitely supported mappings are taken into account. If $G^{(Y)}$ is replaced by G^Y (i.e., the set of all mappings from Y to G with pointwise multiplication), then one speaks of an unrestricted wreath product. For Y finite this makes of course no difference. We will only deal with restricted wreath products. The action of G on X in the permutational wreath product is usually not important for us, but it is nice to have an associative operation. For the right group H, we will only make use of the following cases:

- $H = \mathrm{Sym}(Y)$ acting on Y,
- H a (finite or infinite) cyclic group acting on itself.

Thus, if H is cyclic, the permutational wreath product and the regular wreath product (both denoted by $G \wr H$) coincide. Nevertheless, be aware that $G \wr (H \wr H) = (G \wr H) \wr H$ holds only for the permutational wreath product even if H is cyclic. Note that if G is generated by Σ and H is generated by Γ then $G \wr H$ is generated by $\Sigma \cup \Gamma$.

Leaf-language conventions (Appendix A)

In the following, we introduce basic concepts related to leaf languages, more details can be found in [11, 23, 25, 24, 29]. An NTM M with input alphabet Γ is *adequate*, if (i) for every input $x \in \Gamma^*$, M does not have an infinite computation on input x, (ii) the finite set of transition tuples of M is linearly ordered, and (iii) when terminating M prints a symbol $\alpha(q)$ from a finite alphabet Σ, where q is the current state of M. For an input $x \in \Gamma^*$, we define the computation tree by unfolding the configuration graph of M from the initial configuration. By condition (i) and (ii), the computation tree can be identified with a finite ordered tree $T(x) \subseteq \mathbb{N}^*$. For $u \in T(x)$ let $q(u)$ be the M-state of the configuration that is associated with the tree node u. Then, the leaf string $\mathrm{leaf}(M, x)$ is the string $\alpha(q(v_1)) \cdots \alpha(q(v_k)) \in \Sigma^+$, where $v_1, \ldots, v_k$ are all leaves of $T(x)$ listed in lexicographic order.

An adequate NTM M is called *balanced*, if for every input $x \in \Gamma^*$, $T(x)$ is a complete binary tree. With a language $K \subseteq \Sigma^*$ we associate the language

$$\mathsf{LEAF}(M, K) = \{x \in \Gamma^* \mid \mathrm{leaf}(M, x) \in K\}.$$

Finally, we associate two complexity classes with $K \subseteq \Sigma^*$:

$$\mathsf{LEAF}(K) \quad = \quad \{\mathsf{LEAF}(M, K) \mid M \text{ is an adequate polynomial time NTM}\}$$

$$\mathsf{bLEAF}(K) \quad = \quad \{\mathsf{LEAF}(M, K) \mid M \text{ is a balanced polynomial time NTM}\}$$

These classes are closed under polynomial time reductions. We clearly have $\mathsf{bLEAF}(K) \subseteq \mathsf{LEAF}(K)$. The following result was shown in [29] by padding computation trees to complete binary trees.

Lemma 28. Assume that $K \subseteq \Sigma^*$ is a language such that Σ contains a symbol 1 with the following property: if $uv \in K$ for $u, v \in \Sigma^*$ then $u1v \in K$. Then $\mathsf{LEAF}(K) = \mathsf{bLEAF}(K)$.

In particular, we obtain the following lemma:

Lemma 29. Let G be a finitely generated group and Σ a finite standard generating set for G. Then $\mathsf{LEAF}(\mathrm{WP}(G, \Sigma)) = \mathsf{bLEAF}(\mathrm{WP}(G, \Sigma))$.

Moreover, we have:

Lemma 30. Let G be finitely generated group and Σ, Γ finite standard generating sets for G. Then $\mathsf{LEAF}(\mathrm{WP}(G, \Sigma)) = \mathsf{LEAF}(\mathrm{WP}(G, \Gamma))$.

Proof. Consider a language $L \in \mathsf{LEAF}(\mathrm{WP}(G, \Sigma))$. Thus, there exists an adequate polynomial time NTM M such that $L = \mathsf{LEAF}(M, \mathrm{WP}(G, \Sigma))$. We modify M as follows: If M terminates and prints the symbol $a \in \Sigma$, it enters a small nondeterministic subcomputation that produces the leaf string w_a, where $w_a \in \Gamma^*$ is a word that evaluates to the same group element as a. Let M' be the resulting adequate polynomial time NTM. It follows that $\mathsf{LEAF}(M, \mathrm{WP}(G, \Sigma)) = \mathsf{LEAF}(M', \mathrm{WP}(G, \Gamma))$. □

Lemma 30 allows to omit the standard generating set Σ in the notations $\mathsf{LEAF}(\mathrm{WP}(G, \Sigma))$ and $\mathsf{bLEAF}(\mathrm{WP}(G, \Sigma))$. We will always do that.

For a complexity class $\mathcal{C}$ we denote by $\forall\mathcal{C}$ the class of all languages L such that there exists a polynomial $p(n)$ and a language $K \in \mathcal{C}$ with $L = \{u \mid \forall v \in \{0,1\}^{p(|u|)} : u\#v \in K\}$ (hence, for instance $\forall\mathrm{P} = \mathrm{coNP}$ and $\forall\mathrm{PSPACE} = \mathrm{PSPACE}$).

Straight-line programs and circuit values (Appendix B)

Take a word $w = a_0 \cdots a_{n-1} \in \Sigma^*$ over the alphabet Σ ($n \geq 0$, $a_0, \ldots, a_{n-1} \in \Sigma$). For $0 \leq i < n$ let $w[i] = a_i$ and for $0 \leq i \leq j < n$ let $w[i:j] = a_i a_{i+1} \cdots a_j$. Moreover $w[:i] = w[0:i]$. Note that in the notations $w[i]$ and $w[i:j]$ we take 0 as the first position in w. Finally, with $|w|_a = |\{i \mid a_i = a\}|$ we denote the number of occurrences of a in w. A *straight-line program* (SLP for short) over the alphabet Σ is a triple $\mathcal{G} = (V, \rho, S)$, where V is a finite set of variables such that $V \cap \Sigma = \emptyset$, $S \in V$ is the start variable, and $\rho : V \to (V \cup \Sigma)^*$ is a mapping such that the relation $\{(A, B) \in V \times V : B \text{ occurs in } \rho(A)\}$ is acyclic. For the reader familiar with context free grammars, it might be helpful to view the SLP $\mathcal{G} = (V, \rho, S)$ as the context-free grammar (V, Σ, P, S), where P contains all productions $A \to \rho(A)$ for $A \in V$. The definition of an SLP implies that this context-free grammar derives exactly on terminal word, which will be denoted by $\mathrm{val}(\mathcal{G})$. Formally, one can extend ρ to a morphism $\rho : (V \cup \Sigma)^* \to (V \cup \Sigma)^*$ by setting $\rho(a) = a$ for all $a \in \Sigma$. The above acyclicity condition on ρ implies

that for $m = |V|$ we have $\rho^m(w) \in \Sigma^*$ for all $w \in (V \cup \Sigma)^*$. We then define $\mathrm{val}_{\mathcal{G}}(w) = \rho^m(w)$ (the string derived from the sentential form w) and $\mathrm{val}(\mathcal{G}) = \mathrm{val}_{\mathcal{G}}(S)$.

The word $\rho(A)$ is also called the *right-hand side* of A. Quite often, it is convenient to assume that all right-hand sides are of the form $a \in \Sigma$ or BC with $B, C \in V$. This corresponds to the well-known Chomsky normal form for context-free grammars. There is a simple linear time algorithm that transforms an SLP $\mathcal{G}$ with $\mathrm{val}(\mathcal{G}) \neq \varepsilon$ into an SLP $\mathcal{G}'$ in Chomsky normal form with $\mathrm{val}(\mathcal{G}) = \mathrm{val}(\mathcal{G}')$, see e.g. [38, Proposition 3.8].

We define the size of the SLP $\mathcal{G} = (V, \rho, S)$ as the total length of all right-hand sides: $|\mathcal{G}| = \sum_{A \in V} |\rho(A)|$. SLPs offer a succinct representation of words that contain many repeated substrings. For instance, the word $(ab)^{2^n}$ can be produced by the SLP $\mathcal{G} = (\{A_0, \ldots, A_n\}, \rho, A_n)$ with $\rho(A_0) = ab$ and $\rho(A_{i+1}) = A_i A_i$ for $0 \leq i \leq n-1$. We need the following upper bound on the length of the word $\mathrm{val}(\mathcal{G})$:

Lemma 31 (c.f. [13]). For every SLP $\mathcal{G}$ we have $|\mathrm{val}(\mathcal{G})| \leq 3^{|\mathcal{G}|/3}$.

We also need polynomial time algorithms for a few algorithmic problems for SLPs:

Lemma 32 ([38, Chapter 3]). The following problems can be solved in polynomial time, where $\mathcal{G}$ is an SLP over a terminal alphabet Σ, $a \in \Sigma$, and $p, q \in \mathbb{N}$ (the latter are given in binary notation):

- Given $\mathcal{G}$, compute the length $|\mathrm{val}(\mathcal{G})|$.
- Given $\mathcal{G}$ and a, compute the number $|\mathrm{val}(\mathcal{G})|_a$ of occurrences of a.
- Given $\mathcal{G}$ and p, compute the symbol $\mathrm{val}(\mathcal{G})[p] \in \Sigma$ (in case $0 \leq p < |\mathrm{val}(\mathcal{G})|$ does not hold, the algorithm outputs a special symbol).
- Given $\mathcal{G}$ and p, q, compute an SLP for the string $\mathrm{val}(\mathcal{G})[p : q]$ (in case $0 \leq p \leq q < |\mathrm{val}(\mathcal{G})|$ does not hold, the algorithm outputs a special symbol).

Lemma 33 (c.f. [38, Lemma 3.12]). Given a symbols $a_0 \in \Sigma$ and a sequence of morphisms $\phi_1, \ldots, \phi_n : \Sigma^* \to \Sigma^*$, where every ϕ_i is given by a list of the words $\phi_i(a)$ for $a \in \Sigma$, one can compute in LOGSPACE an SLP for the word $\phi_1(\phi_2(\cdots \phi_n(a_0) \cdots))$.

The *circuit value problem* for a finitely generated group G with the finite standard generating set Σ, $\mathrm{CVP}(G, \Sigma)$ for short, is the following decision problem:

Input: an SLP $\mathcal{G}$ over the alphabet Σ.

Question: does $\mathrm{val}(\mathcal{G}) = 1$ hold in G?

It is an easy observation that the computational complexity of the compressed word problem for G does not depend on the chosen generating set Σ in the sense that if Σ' is another finite standard generating set for G, then $\mathrm{CVP}(G, \Sigma)$ is LOGSPACE-reducible to $\mathrm{CVP}(G, \Sigma')$ [38, Lemma 4.2]. Therefore we do not have to specify the generating set and we just write $\mathrm{CVP}(G)$.

The circuit value problem for G is equivalent to the problem whether a given circuit over the group G evaluates to 1: Take an SLP $\mathcal{G} = (V, \rho, S)$ in Chomsky normal form and built a circuit by taking V is the set of gates. If $\rho(A) = a \in \Sigma$ then A is an input gate that is labelled with the group generator a. If $\rho(A) = BC$ with $B, C \in V$ then B is left input gate for A and C is the right input gate for A. Such a circuit can be evaluated in the natural way (every internal gate computes the product of its input values) and the circuit output is the value at gate S.

Complete Appendix C proof

This section presents a detailed proof of Theorem 6. Instead of circuits over groups we will use the equivalent framework of SLPs from Appendix B. The proof of the lower bound in Theorem 6 uses some of the techniques from the paper [37], where a connection between leaf strings and SLPs was established. In Sections C.1–C.3 we will introduce these techniques. The proof of Theorem 6 will be given in Appendix C.4. Let us first fix some general notations. For integers $i \leq j$ we write $[i..j]$ for the interval $\{z \in \mathbb{Z} \mid i \leq z \leq j\}$. In the following, we will identify a bit string $\alpha = a_1 a_2 \cdots a_n$ $(a_1, \ldots, a_n \in \{0, 1\})$ with the vector $(a_1, a_2, \ldots, a_n)$. In particular, for another vector $\overline{s} = (s_1, \ldots, s_n) \in \mathbb{N}^n$ we will write $\alpha \cdot \overline{s} = \sum_{i=1}^{n} a_i \cdot s_i$ for the scalar product. Moreover, we write $\sum \overline{s}$ for the sum $s_1 + \cdots + s_n$.

C.1 Subsetsum problems

A sequence $(s_1, \ldots, s_n)$ of natural numbers is *super-decreasing* if $s_i > s_{i+1} + \cdots + s_n$ for all $i \in [1..n]$. For example, $(s_1, \ldots, s_n)$ with $s_i = 2^{n-i}$ is super-decreasing. An instance of the *subsetsum problem* is a tuple $(t, s_1, \ldots, s_k)$ of binary coded natural numbers. It is a positive instance if there are $a_1, \ldots, a_k \in \{0, 1\}$ such that $t = a_1 s_1 + \cdots + a_k s_k$. Subsetsum is a classical NP-complete problem, see e.g. [15]. The *super-decreasing subsetsum* problem is the restriction of subsetsum to instances $(t, s_1, \ldots, s_k)$, where $(s_1, \ldots, s_k)$ is super-decreasing. In [30] it was shown that super-decreasing subsetsum is P-complete.[1] We need a slightly generalized version of the construction showing P-hardness that we discuss in Appendix C.2.

C.2 From Boolean circuits to super-decreasing subsetsum

For this section, we have to fix some more details on boolean circuits. Let us consider a boolean circuit C with input gates $x_1, \ldots, x_m$ and output gates $y_0, \ldots, y_{n-1}$.[2] We view C as a directed acyclic graph with multi-edges (there can be two edges between two nodes); the nodes are the gates of the circuit. The number of incoming edges of a gate is called its *fan-in* and the number of outgoing edges is the *fan-out*. Every input gate x_i has fan-in zero and every output gate y_i has fan-out zero. Besides the input gates there are two more gates

[1] In fact, [30] deals with the *super-increasing* subsetsum problem. But this is only a nonessential detail. For our purpose, super-decreasing sequences are more convenient.

[2] It will be convenient for us to number the input gates from 1 and the output gates from 0.

c_0 and c_1 of fan-in zero, where c_i carries the constant truth value $i \in \{0,1\}$. Besides $x_1, \ldots, x_m, c_0, c_1$ every other gate has fan-in two and computes the nand of its two input gates. Moreover, we assume that every output gate y_i is a nand-gate. For a bit string $\alpha = b_1 \cdots b_m$ ($b_1, \ldots, b_m \in \{0,1\}$) and $0 \leq i \leq n-1$ we denote with $C(\alpha)_i$ the value of the output gate y_i when every input gate x_j ($1 \leq j \leq m$) is set to b_j. Thus, C defines a map $\{0,1\}^m \to \{0,1\}^n$.

We assume now that C is a boolean circuit as above with the following additional property that will be satisfied later: For all input bit strings $\alpha \in \{0,1\}^m$ there is exactly one $i \in [0..n-1]$ such that $C(\alpha)_i = 1$. Using a refinement of the construction from [30] we compute in LOGSPACE $q_0, \ldots, q_{n-1} \in \mathbb{N}$ and two super-decreasing sequences $\overline{r} = (r_1, \ldots r_m)$ and $\overline{s} = (s_1, \ldots, s_k)$ for some k (all numbers are represented in binary notation) with the following properties:

- The $r_1, \ldots, r_m$ are pairwise distinct powers of 4.
- For all $0 \leq i \leq n-1$ and all $\alpha \in \{0,1\}^m$: $C(\alpha)_i = 1$ if and only if there exists $\delta \in \{0,1\}^k$ such that $\delta \cdot \overline{s} = q_i + \alpha \cdot \overline{r}$.

Let us first add for every input gate x_i two new nand-gates $\bar{x}_i$ and $\bar{\bar{x}}_i$, where $\bar{\bar{x}}_i$ has the same outgoing edges as x_i. Moreover we remove the old outgoing edges of x_i and replace them by the edges $(x_i, \bar{x}_i)$, $(c_1, \bar{x}_i)$ and two edges from $\bar{x}_i$ to $\bar{\bar{x}}_i$. This has the effect that every input gate x_i has a unique outgoing edge. Clearly, the new circuit computes the same boolean function (basically, we introduce two negation gates for every input gate). Let $g_1, \ldots, g_p$ be the nand-gates of the circuit enumerated in reverse topological order, i.e., if there is an edge from gate g_i to gate g_j then $i > j$. We denote the two edges entering gate g_i with e_{2i+n-2} and e_{2i+n-1}. Moreover, we write e_i ($0 \leq i \leq n-1$) for an imaginary edge that leaves the output gate y_i and whose target gate is unspecified. Thus, the edges of the circuit are $e_0, \ldots, e_{2p+n-1}$. We now define the natural numbers $q_0, \ldots, q_{n-1}, r_1, \ldots r_m, s_1, \ldots, s_k$ with $k = 3p$:

- Let $I = \{j \mid e_j \text{ is an outgoing edge of the constant gate } c_1 \text{ or a nand-gate}\}$. For $0 \leq i \leq n-1$ we define the number q_i as

$$q_i = \sum_{j \in I \setminus \{i\}} 4^j.$$

  Recall that e_i is the unique outgoing edge of the output gate y_i.

- If e_j is the unique outgoing edge of the input gate x_i then we set $r_i = 4^j$. We can choose the reverse topological sorting of the nand-gates in such a way that $r_1 > r_2 > \cdots > r_m$ (we only have to ensure that the target gates $\overline{x}_1, \ldots, \overline{x}_m$ of the input gates appear in the order $\overline{x}_m, \ldots, \overline{x}_1$ in the reverse topological sorting of the nand-gates).

- To define the numbers $s_1, \ldots, s_k$ we first define for every nand-gate g_i three numbers t_{3i}, t_{3i-1} and t_{3i-2} as follows, where $I_i = \{j \mid e_j \text{ is an outgoing edge of gate } g_i\}$:

$$\begin{aligned} t_{3i} &= 4^{2i+n-1} + 4^{2i+n-2} + \sum_{j \in I_i} 4^j \\ t_{3i-1} &= 4^{2i+n-1} - 4^{2i+n-2} = 3 \cdot 4^{2i+n-2} \\ t_{3i-2} &= 4^{2i+n-2} \end{aligned}$$

Then, the tuple $(s_1, \ldots, s_k)$ is $(t_{3p}, t_{3p-1}, t_{3p-2}, \ldots, t_3, t_2, t_1)$, which is indeed super-decreasing (see also [30]). In fact, we have $s_i - (s_{i+1} + \cdots + s_k) \geq 4^{n-1}$ for all $i \in [1..k]$. To see this, note that the sets $I_{i+1}, \ldots, I_k$ are pairwise disjoint. This implies that the $n-1$ low-order digits in the base-4 expansion of $s_{i+1} + \cdots + s_k$ are zero or one.

In order to understand this construction, one should think of the edges of the circuit carrying truth values. Recall that there are $2p+n$ edges in the circuit (including the imaginary outgoing edges of the output gates $y_0, \ldots, y_{n-1}$). A number in base-4 representation with $2p+n$ digits that are either 0 or 1 represents a truth assignment to the $2p+n$ edges, where a 1-digit represents the truth value 1 and a 0-digit represents the truth value 0. Consider an input string $\alpha = b_1 \cdots b_m \in \{0,1\}^m$ and consider an output gate y_i, $i \in [0..n-1]$. Then the number $N := 4^i + q_i + b_1 r_1 + \cdots + b_m r_m$ encodes the truth assignment for the circuit edges, where:

- all outgoing edges of the constant gate c_1 carry the truth value 1,
- all outgoing edges of the constant gate c_0 carry the truth value 0,
- the unique outgoing edge of an input gate x_i carries the truth value b_i,
- all outgoing edges of nand-gates carry the truth value 1.

We have to show that $C(\alpha)_i = 1$ if and only if there exists $\delta \in \{0,1\}^k$ such that $\delta \cdot \overline{s} = N - 4^i$. For this we apply the canonical algorithm for super-decreasing subsetsum with input $(N, s_1, \ldots, s_k)$. This algorithm initializes a counter A to N and then goes over the sequence $s_1, \ldots, s_k$ in that order. In the j-th step $(1 \leq j \leq k)$ we set A to $A - s_j$ if $A \geq s_j$. If $A < s_j$ then we do not modify A. After that we proceed with s_{j+1}. The point is that this process simulates the evaluation of the circuit on the input values $b_1, \ldots, b_m$. Thereby the nand-gates are evaluated in the topological order $g_p, g_{p-1}, \ldots, g_1$. Assume that g_j is the gate that we want to evaluate next. In the above algorithm for super-decreasing subsetsum the evaluation of g_j is simulated by the three numbers t_{3j}, t_{3j-1}, and t_{3j-2}. At the point where the algorithm checks whether t_{3j} can be subtracted from A, the base-4 digits at positions $2j+n, \ldots, 2p+n-1$ in the counter value A have been already set to zero. If the digits at the next two high-order positions $2j+n-1$ and $2j+n-2$ are still 1 (i.e., the input edges e_{2j+n-2} and e_{2j+n-1} for gate g_j carry the truth value 1), then we can subtract t_{3j} from A. Thereby we subtract all powers 4^{2j+n-1}, 4^{2j+n-2} and 4^h, where e_h is an outgoing edge for gate g_j. Since gate g_j evaluates to zero (both input edges carry 1), this subtraction correctly simulates the evaluation of gate g_j: all outgoing edges e_h of g_j (that were initially set to the truth value 1) are set to the truth value 0. On the other hand, if one of the two digits at positions $2j+n-1$ and $2j+n-2$ in A is 0 (which means that gate g_j evaluates to 1), then we cannot subtract t_{3j} from A. If both digits at positions $2j+n-1$ and $2j+n-2$ in A are 0, then also t_{3j-1} and t_{3j-2} cannot be subtracted. On the other hand, if exactly one of the two digits at positions $2j+n-1$ and $2j+n-2$ is 1, then with t_{3j-1} and t_{3j-2} we can set these two digits to 0 (thereby digits at positions $< 2j+n-2$ are not modified).

Assume now that y_j ($j \in [0..n-1]$) is the unique output gate that evaluates to 1, i.e., all output gates $y_{j'}$ with $j' \neq j$ evaluate to zero. Then after processing

all weights $s_1, \ldots, s_k$ we have $A = 4^j$ (we will never subtract 4^j). We have shown that there exists $\delta \in \{0,1\}^k$ such that $\delta \cdot \overline{s} + 4^j = N$. Hence, if $i = j$ (i.e., $C(\alpha)_i = 1$) then $\delta \cdot \overline{s} = N - 4^i$. Now assume that $i \neq j$. In order to get a contradiction assume that there is $\delta' \in \{0,1\}^k$ such that $\delta' \cdot \overline{s} + 4^i = N$. We have $\delta \neq \delta'$ and $\delta \cdot \overline{s} + 4^j = \delta' \cdot \overline{s} + 4^i$, i.e, $\delta \cdot \overline{s} - \delta' \cdot \overline{s} = 4^i - 4^j$. Since $i, j \in [0..n-1]$ we get $|\delta \cdot \overline{s} - \delta' \cdot \overline{s}| < 4^{n-1}$. But $s_i - (s_{i+1} + \cdots s_k) \geq 4^{n-1}$ for all $i \in [1..k]$ implies that $|\delta \cdot \overline{s} - \delta' \cdot \overline{s}| \geq 4^{n-1}$.

C.3 From super-decreasing subsetsum to straight-line programs

In [35] a super-decreasing sequence $\overline{t} = (t_1, \ldots, t_k)$ of natural numbers is encoded by the string $S(\overline{t}) \in \{0,1\}^*$ of length $\sum \overline{t} + 1$ such that for all $0 \leq p \leq \sum \overline{t}$:

$$S(\overline{t})[p] = \begin{cases} 1 & \text{if } p = \alpha \cdot \overline{t} \text{ for some } \alpha \in \{0,1\}^k, \\ 0 & \text{otherwise.} \end{cases} \tag{3}$$

Note that in the first case, α is unique. Since $\overline{t}$ is a super-decreasing sequence, the number of 1's in the string $S(\overline{t})$ is 2^k. Also note that $S(\overline{t})$ starts and ends with 1. In [35] it was shown that from a super-decreasing sequence $\overline{t}$ of binary encoded numbers one can construct in LOGSPACE an SLP for the word $S(\overline{t})$.

C.4 Proof of Theorem 6

Let us fix a regular wreath product of the form $G \wr \mathbb{Z}$ for a finitely generated group G. Such groups are also known as generalized lamplighter groups (the lamplighter group arises for $G = \mathbb{Z}_2$). Throughout this section, we fix a set of standard generators Σ for G and let $\tau = 1$ be the generator for $\mathbb{Z}$. Then $\Sigma \cup \{\tau, \tau^{-1}\}$ is a standard generating set for the wreath product $G \wr \mathbb{Z}$. In $G \wr \mathbb{Z}$ the G-generator $a \in \Sigma$ represents the mapping $f_a \in G^{(\mathbb{Z})}$ with $f_a(0) = a$ and $f_a(z) = 1$ for $z \neq 0$. For a word $w \in (\Sigma \cup \{\tau, \tau^{-1}\})^*$ we define $\eta(w) := |w|_\tau - |w|_{\tau^{-1}}$. Thus, the element of $G \wr \mathbb{Z}$ represented by w is of the form $f\tau^{\eta(w)}$ for some $f \in G^{(\mathbb{Z})}$. Recall the definition of the left action of $\mathbb{Z}$ on $G^{(\mathbb{Z})}$ from the wreath-product conventions (where we take $H = Y = \mathbb{Z}$). For better readability, we write $c \circ f$ for ${}^c f$ ($c \in \mathbb{Z}$, $f \in G^{(\mathbb{Z})}$). Hence, we have $(c \circ f)(z) = f(z + c)$. If one thinks of f as a bi-infinite word over the alphabet G, then $c \circ f$ is the same word but shifted by $-c$.

The following intuition might be helpful: Consider a word $w \in (\Sigma \cup \{\tau, \tau^{-1}\})^*$. In $G \wr \mathbb{Z}$ we can simplify w to a word of the form $\tau^{z_0} a_1 \tau^{z_1} a_2 \cdots \tau^{z_{k-1}} a_k \tau^{z_k}$ (with $z_j \in \mathbb{Z}$, $a_j \in \Sigma$), which in $G \wr \mathbb{Z}$ can be rewritten as

$$\tau^{z_0} a_1 \tau^{z_1} a_2 \cdots \tau^{z_{k-1}} a_k \tau^{z_k} = \Big(\prod_{j=1}^{k} \tau^{z_0 + \cdots + z_{j-1}} a_j \tau^{-(z_0 + \cdots + z_{j-1})} \Big) \, \tau^{z_0 + \cdots + z_k}.$$

Hence, the word w represents the group element

$$\Big(\prod_{j=1}^{k} (z_0 + \cdots + z_{j-1}) \circ f_{a_j} \Big) \, \tau^{z_0 + \cdots + z_k}.$$

This gives the following intuition for evaluating $\tau^{z_0} a_1 \tau^{z_1} a_2 \cdots \tau^{z_{k-1}} a_k \tau^{z_k}$: In the beginning, every $\mathbb{Z}$-position carries the G-value 1. First, go to the $\mathbb{Z}$-position $-z_0$ and multiply the G-element at this position with a_1 (on the right), then go to the $\mathbb{Z}$-position $-z_0 - z_1$ and multiply the G-element at this position with a_2, and so on.

Proof of Theorem 6. The easy part is to show that the circuit value problem for $G \wr \mathbb{Z}$ belongs to $\forall\mathsf{LEAF}(\mathrm{WP}(G))$. In the following, we make use of the statements from Lemma 32. Let $\mathcal{G}$ be an SLP over the alphabet $\Sigma \cup \{\tau, \tau^{-1}\}$ and let $f\tau^{\eta(\mathrm{val}(\mathcal{G}))} \in G \wr \mathbb{Z}$ be the group element represented by $\mathrm{val}(\mathcal{G})$. By Lemma 32 we can compute $\eta(\mathrm{val}(\mathcal{G}))$ in polynomial time. If $\eta(\mathrm{val}(\mathcal{G})) \neq 0$ then the Turing-machine rejects by printing a non-trivial generator of G (here we need the assumption that G is non-trivial). So, let us assume that $\eta(\mathrm{val}(\mathcal{G})) = 0$. We can also compute in polynomial time two integers $b, c \in \mathbb{Z}$ such that $\mathrm{supp}(f) \subseteq [b..c]$. We can take for instance $b = -|\mathrm{val}(\mathcal{G})|$ and $c = |\mathrm{val}(\mathcal{G})|$. It suffices to check whether for all $x \in [b..c]$ we have $f(x) = 1$. For this, the Turing-machine branches universally to all binary coded integers $x \in [b..c]$ (this yields the $\forall$-part in $\forall\mathsf{LEAF}(\mathrm{WP}(G))$). Consider a specific branch that leads to the integer $x \in [b..c]$. From x and the input SLP $\mathcal{G}$ the Turing-machine then produces a leaf string over the standard generating set Σ of G such that this leaf string represents the group element $f(x) \in G$. For this, the machine branches to all positions $p \in [0..|\mathrm{val}(\mathcal{G})| - 1]$ (if $p < q < |\mathrm{val}(\mathcal{G})|$ then the branch for p is to the left of the branch for q). For a specific position p, the machine computes in polynomial time the symbol $a = \mathrm{val}(\mathcal{G})[p]$. If a is τ or τ^{-1} then the machine prints $1 \in \Sigma$. On the other hand, if $a \in \Sigma$ then the machine computes in polynomial time $d = \eta(\mathrm{val}(\mathcal{G})[: p])$. This is possible by first computing an SLP for the prefix $\mathrm{val}(\mathcal{G})[: p]$. If $d = -x$ then the machine prints the symbol a, otherwise the machine prints the trivial generator 1. It is easy to observe that the leaf string produced in this way represents the group element $f(x)$.

We now show the hardness statement from Theorem 6. By Lemma 29 it suffices to show that $\mathrm{CVP}(G \wr \mathbb{Z})$ is hard for $\forall\mathsf{bLEAF}(\mathrm{WP}(G/Z(G)))$ with respect to $\mathsf{LOGSPACE}$-reductions. Let $a_0, \ldots, a_{n-1}$ be an arbitrary enumeration of the standard generators in Σ. Fix a language $L \in \forall\mathsf{bLEAF}(\mathrm{WP}(G/Z(G)))$. From the definition of the class $\forall\mathsf{bLEAF}(\mathrm{WP}(G/Z(G)))$ it follows that there exist two polynomials p_1 and p_2 and a balanced polynomial time NTM M running in time $p_1 + p_2$ that outputs a symbol from Σ after termination and such that the following holds: Consider an input word z and let $T(z)$ be the corresponding computation tree of M. Let $m_1 = p_1(|z|)$, $m_2 = p_2(|z|)$, and $m = m_1 + m_2$. Note that the nodes of $T(z)$ are the bit strings of length at most m. For every leaf $\alpha \in \{0,1\}^m$ let us denote with $\lambda(\alpha)$ the symbol from Σ that M prints when reaching the leaf α. Then $z \in L$ if and only if for all $\beta \in \{0,1\}^{m_1}$ the string

$$\lambda_\beta := \prod_{\gamma \in \{0,1\}^{m_2}} \lambda(\beta\gamma) \tag{4}$$

represents a group element from the center $Z(G)$. Here (and in the following), the product in the right-hand side of (4) goes over all bit strings of length m_2 in lexicographic order. Our construction consists of five steps:

Step 1. Note that given a bit string $\alpha \in \{0,1\}^m$, we can compute in polynomial time the symbol $\lambda(\alpha) \in \Sigma$ by following the computation path specified by α.

Using the classical Cook-Levin construction (see e.g. [2]), we can compute from the input z and $a \in \Sigma$ in LOGSPACE a boolean circuit $C_{z,a}$ with m input gates $x_1, \ldots, x_m$ and a single output gate y_0 such that for all $\alpha \in \{0,1\}^m$: $C_{z,a}(\alpha)_0 = 1$ if and only if $\lambda(\alpha) = a$. By taking the disjoint union of these circuits and merging the input gates, we can build a single circuit C_z with m input gates $x_1, \ldots, x_m$ and $n = |\Sigma|$ output gates $y_0, \ldots, y_{n-1}$. For every $\alpha \in \{0,1\}^m$ and every $0 \leq i \leq n-1$ the following holds: $C_z(\alpha)_i = 1$ if and only if $\lambda(\alpha) = a_i$.

Step 2. Using the construction from Appendix C.2 we can compute from the circuit C_z in LOGSPACE numbers $q_0, \ldots, q_{n-1} \in \mathbb{N}$ and two super-decreasing sequences $\overline{r} = (r_1, \ldots, r_m)$ and $\overline{s} = (s_1, \ldots, s_k)$ with the following properties:

- The $r_1, \ldots, r_m$ are pairwise distinct powers of 4.
- For all $0 \leq i \leq n-1$ and all $\alpha \in \{0,1\}^m$ we have: $\lambda(\alpha) = a_i$ if and only if $C_z(\alpha)_i = 1$ if and only if there is $\delta \in \{0,1\}^k$ such that $\delta \cdot \overline{s} = q_i + \alpha \cdot \overline{r}$.

Note that for all $\alpha \in \{0,1\}^m$ there is a unique i such that $C_z(\alpha)_i = 1$. Hence, for all $\alpha \in \{0,1\}^m$ there is a unique i such that $q_i + \alpha \cdot \overline{r}$ is of the form $\delta \cdot \overline{s}$ for some $\delta \in \{0,1\}^k$. For this unique i we have $\lambda(\alpha) = a_i$.

We split the super-decreasing sequence $\overline{r} = (r_1, \ldots, r_m)$ into the two sequences $\overline{r}_1 = (r_1, \ldots, r_{m_1})$ and $\overline{r}_2 = (r_{m_1+1}, \ldots, r_m)$. For the following consideration, we need the following numbers:

$$\ell = \max\left\{ \sum \overline{r}_1 + \max\{q_0, \ldots, q_{n-1}\} + 1,\ \sum \overline{s} - \sum \overline{r}_2 - \min\{q_0, \ldots, q_{n-1}\} + 1 \right\} \quad (5)$$

$$\pi = \ell + \sum \overline{r}_2 \quad (6)$$

The binary codings of these numbers can be computed in LOGSPACE (since iterated addition, max, and min can be computed in LOGSPACE). The precise value of ℓ will be only relevant at the end of step 4.

Step 3. By the result from [35] (see Section) we can construct in LOGSPACE from the three super-decreasing sequences $\overline{r}_1, \overline{r}_2$ and $\overline{s}$ three SLPs $\mathcal{G}_1$, $\mathcal{G}_2$ and $\mathcal{H}$ over the alphabet $\{0,1\}$ such that $\mathrm{val}(\mathcal{G}_1) = S(\overline{r}_1)$, $\mathrm{val}(\mathcal{G}_2) = S(\overline{r}_2)$ and $\mathrm{val}(\mathcal{H}) = S(\overline{s})$ (see (3)). For all positions $p \geq 0$ (in the suitable range) we have:

$$\begin{aligned} \mathrm{val}(\mathcal{G}_1)[p] = 1 &\iff \exists \beta \in \{0,1\}^{m_1} : p = \beta \cdot \overline{r}_1 \\ \mathrm{val}(\mathcal{G}_2)[p] = 1 &\iff \exists \gamma \in \{0,1\}^{m_2} : p = \gamma \cdot \overline{r}_2 \\ \mathrm{val}(\mathcal{H})[p] = 1 &\iff \exists \delta \in \{0,1\}^{k} : p = \delta \cdot \overline{s} \end{aligned}$$

Note that $|\mathrm{val}(\mathcal{G}_1)| = \sum \overline{r}_1 + 1$, $|\mathrm{val}(\mathcal{G}_2)| = \sum \overline{r}_2 + 1$, and $|\mathrm{val}(\mathcal{H})| = \sum \overline{s} + 1$.

Step 4. We build in LOGSPACE for every $i \in [0..n-1]$ an SLP $\mathcal{H}_i$ from the SLP $\mathcal{H}$ by replacing in every right-hand side of $\mathcal{H}$ every occurrence of 0 by τ^{-1} and every occurrence of 1 by $a_i\tau^{-1}$. Let T_i be the start variable of $\mathcal{H}_i$, let S_1 be the start variable of $\mathcal{G}_1$, and let S_2 be the start variable of $\mathcal{G}_2$. We can assume that the variable sets of the SLPs $\mathcal{G}_1, \mathcal{G}_2, \mathcal{H}_0, \ldots, \mathcal{H}_{n-1}$ are pairwise disjoint. We next combine these SLPs into a single SLP $\mathcal{I}$. The variables of $\mathcal{I}$ are the variables of the SLPs $\mathcal{G}_1, \mathcal{G}_2, \mathcal{H}_0, \ldots, \mathcal{H}_{n-1}$ plus a fresh variable S which is the start variable of $\mathcal{I}$. The right-hand sides for the variables are defined below. In the right-hand sides we write powers τ^p for integers p whose binary codings can be computed in LOGSPACE. Such powers can be produced by small subSLPs that can be constructed in LOGSPACE too.

- In all right-hand sides of $\mathcal{G}_1$ and $\mathcal{G}_2$ we replace all occurrences of the terminal symbol 0 by the $\mathbb{Z}$-generator τ.
- We replace every occurrence of the terminal symbol 1 in a right-hand side of $\mathcal{G}_1$ by $S_2\tau^\ell$, where ℓ is from (5).
- We replace every occurrence of the terminal symbol 1 in a right-hand side of $\mathcal{G}_2$ by $\sigma\tau$, where

$$\sigma = \tau^{q_0} T_0 \tau^{h-q_0} \tau^{q_1} T_1 \tau^{h-q_1} \cdots \tau^{q_{n-1}} T_{n-1} \tau^{h-q_{n-1}} \tag{7}$$

  and $h = \sum \overline{s} + 1$ is the length of the word $\mathrm{val}(\mathcal{H})$ (which is $-\eta(\mathrm{val}_{\mathcal{I}}(T_i))$ for every $i \in [0..n-1]$). Note that $\eta(\mathrm{val}_{\mathcal{I}}(\sigma)) = 0$.
- Finally, the right-hand side of the start variable S is $S_1\tau^{-d}$ where $d := \sum \overline{r}_1 + 1 + 2^{m_1} \cdot \pi$. (note that $d = \eta(\mathrm{val}_{\mathcal{I}}(S_1))$).

Before we explain this construction, let us first introduce some notations.

- Let $u := \mathrm{val}_{\mathcal{I}}(S_2)$. We have $\eta(u) = |\mathrm{val}(\mathcal{G}_2)|$. Hence, the group element represented by u can be written as $f_u\tau^{|\mathrm{val}(\mathcal{G}_2)|}$ for a mapping $f_u \in G^{(\mathbb{Z})}$.
- Let $v := \mathrm{val}_{\mathcal{I}}(\sigma)$ where σ is from (7). Note that $\eta(v) = 0$. Hence, the group element represented by v is a mapping $f_v \in G^{(\mathbb{Z})}$. Its support is a subset of the interval from position $-\max\{q_0, \ldots, q_{n-1}\}$ to position $\sum \overline{s} - \min\{q_0, \ldots, q_{n-1}\}$.
- For $\beta \in \{0,1\}^{m_1}$ let $\mathsf{bin}(\beta)$ be the number represented by β in binary notation (thus, $\mathsf{bin}(0^{m_1}) = 0$, $\mathsf{bin}(0^{m_1-1}1) = 1$, ..., $\mathsf{bin}(1^{m_1}) = 2^{m_1} - 1$). Moreover, let

$$p_\beta := -\mathsf{bin}(\beta) \cdot \pi.$$

First, note that $\eta(\mathrm{val}(\mathcal{I})) = 0$. This is due to the factor τ^{-d} in the right-hand side of the start variable S of $\mathcal{I}$. Hence, the group element represented by $\mathrm{val}(\mathcal{I})$ is a mapping $f \in G^{(\mathbb{Z})}$. The crucial claim is the following:

Claim. For every $\beta \in \{0,1\}^{m_1}$, $f(p_\beta)$ is the group element represented by the leaf string λ_β from (4).

Proof of the claim. In the following, we compute in the restricted direct product $G^{(\mathbb{Z})}$. Recall that the multiplication in this group is defined by the pointwise multiplication of mappings.

Since we replaced in $\mathcal{G}_1$ every 1 in a right-hand side by $S_2\tau^\ell$, which produces $u\tau^\ell$ in $\mathcal{I}$ (which evaluates to $f_u\tau^{\pi+1}$) the mapping f is a product (in the restricted direct product $G^{(\mathbb{Z})}$) of shifted copies of f_u. More precisely, for every $\beta' \in \{0,1\}^{m_1}$ we get the shifted copy

$$\big(\beta' \cdot \overline{r}_1 + \mathsf{bin}(\beta') \cdot \pi\big) \circ f_u \tag{8}$$

of f_u. The shift distance $\beta' \cdot \overline{r}_1 + \mathsf{bin}(\beta') \cdot \pi$ can be explained as follows: The 1 in $\mathrm{val}(\mathcal{G}_1)$ that corresponds to $\beta' \in \{0,1\}^{m_1}$ occurs at position $\beta' \cdot \overline{r}_1$ (the first position is 0) and to the left of this position we find $\mathsf{bin}(\beta')$ many 1's and $\beta' \cdot \overline{r}_1 - \mathsf{bin}(\beta')$ many 0's in $\mathrm{val}(\mathcal{G}_1)$. Moreover, every 0 in $\mathrm{val}(\mathcal{G}_1)$ was

replaced by τ (shift by 1) and every 1 in $\mathrm{val}(\mathcal{G}_1)$ was replaced by $u\tau^\ell$ (shift by $\ell + |\mathrm{val}(\mathcal{G}_2)| = \pi + 1$). Hence, the total shift distance is indeed (8). Also note that if $\beta' \in \{0,1\}^{m_1}$ is lexicographically smaller than $\beta'' \in \{0,1\}^{m_1}$ then $\beta' \cdot \overline{r}_1 < \beta'' \cdot \overline{r}_1$. This implies that

$$f = \prod_{\beta' \in \{0,1\}^{m_1}} \big(\beta' \cdot \overline{r}_1 + \mathsf{bin}(\beta') \cdot \pi\big) \circ f_u = \prod_{\beta' \in \{0,1\}^{m_1}} \big(\beta' \cdot \overline{r}_1 - p_{\beta'}\big) \circ f_u.$$

Let us now compute the mapping f_u. Recall that we replaced in $\mathcal{G}_2$ every occurrence of 1 by $\sigma\tau$, where σ is from (7) and derives to v. The 1's in $\mathrm{val}(\mathcal{G}_2)$ occur at positions of the form $\gamma \cdot \overline{r}_2$ for $\gamma \in \{0,1\}^{m_2}$ and if $\gamma \in \{0,1\}^{m_2}$ is lexicographically smaller than $\gamma' \in \{0,1\}^{m_2}$ then $\gamma \cdot \overline{r}_2 < \gamma' \cdot \overline{r}_2$. We therefore get

$$f_u = \prod_{\gamma \in \{0,1\}^{m_2}} (\gamma \cdot \overline{r}_2) \circ f_v.$$

We obtain

$$\begin{aligned} f &= \prod_{\beta' \in \{0,1\}^{m_1}} \big(\beta' \cdot \overline{r}_1 - p_{\beta'}\big) \circ f_u \\ &= \prod_{\beta' \in \{0,1\}^{m_1}} \big(\beta' \cdot \overline{r}_1 - p_{\beta'}\big) \circ \prod_{\gamma \in \{0,1\}^{m_2}} (\gamma \cdot \overline{r}_2 \circ f_v) \\ &= \prod_{\beta' \in \{0,1\}^{m_1}} \prod_{\gamma \in \{0,1\}^{m_2}} \big(\beta' \cdot \overline{r}_1 + \gamma \cdot \overline{r}_2 - p_{\beta'}\big) \circ f_v \end{aligned}$$

and hence

$$f(p_\beta) = \prod_{\beta' \in \{0,1\}^{m_1}} \prod_{\gamma \in \{0,1\}^{m_2}} f_v(p_\beta - p_{\beta'} + \beta' \cdot \overline{r}_1 + \gamma \cdot \overline{r}_2).$$

We claim that for all $\beta \neq \beta'$ and all $\gamma \in \{0,1\}^{m_2}$ we have

$$f_v(p_\beta - p_{\beta'} + \beta' \cdot \overline{r}_1 + \gamma \cdot \overline{r}_2) = 1. \tag{9}$$

Let us postpone the proof of this for a moment. From (9) we get

$$f(p_\beta) = \prod_{\gamma \in \{0,1\}^{m_2}} f_v(\beta \cdot \overline{r}_1 + \gamma \cdot \overline{r}_2).$$

Consider a specific $\gamma \in \{0,1\}^{m_2}$ and let $\alpha = \beta\gamma$ and $p = \beta \cdot \overline{r}_1 + \gamma \cdot \overline{r}_2 = \alpha \cdot \overline{r}$. From the definition of $v = \mathrm{val}_{\mathcal{I}}(\sigma)$ it follows that for all $x \in \mathbb{Z}$, $f_v(x)$ is a product of those group generators a_i such that $x = -q_i + \delta \cdot \overline{s}$ for some $\delta \in \{0,1\}^k$. For the position p this means that $q_i + \alpha \cdot \overline{r} = \delta \cdot \overline{s}$. By our previous remarks, there is a unique such $i \in [0..n-1]$ and for this i we have $\lambda(\alpha) = a_i$. Hence, we obtain $f_v(p) = \lambda(\alpha) = \lambda(\beta\gamma)$ and thus

$$f(p_\beta) = \prod_{\gamma \in \{0,1\}^{m_2}} \lambda(\beta\gamma) = \lambda_\beta.$$

It remains to show (9). To get this identity, we need the precise value of ℓ from (5) (so far, the value of ℓ was not relevant). Assume now that $\beta \neq \beta'$, which implies

$$|p_\beta - p_{\beta'}| \geq \pi = \ell + \sum \overline{r}_2.$$

Hence, we either have

$$\begin{aligned} p_\beta - p_{\beta'} + \beta' \cdot \overline{r}_1 + \gamma \cdot \overline{r}_2 &\geq \ell + \sum \overline{r}_2 + \beta' \cdot \overline{r}_1 + \gamma \cdot \overline{r}_2 \\ &\geq \ell + \sum \overline{r}_2 \\ &> \sum \overline{s} - \min\{q_0, \ldots, q_{n-1}\} \end{aligned}$$

or

$$\begin{aligned} p_\beta - p_{\beta'} + \beta' \cdot \overline{r}_1 + \gamma \cdot \overline{r}_2 &\leq -\ell - \sum \overline{r}_2 + \beta' \cdot \overline{r}_1 + \gamma \cdot \overline{r}_2 \\ &\leq -\ell + \sum \overline{r}_1 \\ &< -\max\{q_0, \ldots, q_{n-1}\}, \end{aligned}$$

where the strict inequalities follow from our choice of ℓ. Recall that the support of the mapping f_v is contained in $[-\max\{q_0, \ldots, q_{n-1}\}..\sum \overline{s} - \min\{q_0, \ldots, q_{n-1}\}]$. This shows (9) and hence the claim.

Step 5. By the above claim, we have $f(p_\beta) \in Z(G)$ for all $\beta \in \{0,1\}^{m_1}$ if and only if $\lambda_\beta \in Z(G)$ for all $\beta \in \{0,1\}^{m_1}$, which is equivalent to $z \in L$. The only remaining problem is that the word $\mathrm{val}(\mathcal{I})$ produces some "garbage" group elements $f(x)$ on positions x that are not of the form p_β. Note that for every $g \in G \setminus Z(G)$, there is a generator $a_i \in \Sigma$ such that the commutator $[g, a_i]$ is non-trivial. We now produce from $\mathcal{I}$ an SLP $\mathcal{I}^{-1}$ such that $\mathrm{val}(\mathcal{I}^{-1})$ represents the inverse element of $f \in G^{(\mathbb{Z})}$, which is the mapping g with $g(x) = f(x)^{-1}$ for all $x \in \mathbb{Z}$. To construct $\mathcal{I}^{-1}$, we have to reverse every right-hand side of $\mathcal{I}$ and replace every occurrence of a symbol $a_0, \ldots, a_{n-1}, \tau, \tau^{-1}$ by its inverse.

It is easy to compute in LOGSPACE for every $i \in [0..n-1]$ an SLP for the word

$$w_i := \left(a_i \tau^\pi\right)^{2^{m_1}} \tau^{-2^{m_1} \cdot \pi}.$$

Then the group element represented by w_i is the mapping $f_i \in G^{(\mathbb{Z})}$ whose support is the set of positions p_β for $\beta \in \{0,1\}^{m_1}$ and $f_i(p_\beta) = a_i$ for all $\beta \in \{0,1\}^{m_1}$. We can also compute in LOGSPACE an SLP for the word w_i^{-1}. We then built in LOGSPACE SLPs $\mathcal{J}_0, \ldots, \mathcal{J}_{n-1}$ such that $\mathrm{val}(\mathcal{J}_i) = \mathrm{val}(\mathcal{I}^{-1}) w_i^{-1} \mathrm{val}(\mathcal{I}) w_i$. Hence, the word $\mathrm{val}(\mathcal{J}_i)$ represents the group element $g_i \in G^{(\mathbb{Z})}$, where $g_i(x) = 1$ for all $x \in \mathbb{Z} \setminus \{p_\beta \mid \beta \in \{0,1\}^{m_1}\}$ and $g_i(p_\beta) = f(p_\beta)^{-1} a_i^{-1} f(p_\beta) a_i = [f(p_\beta), a_i]$.

Finally, we construct in LOGSPACE an SLP $\mathcal{J}$ such that

$$\mathrm{val}(\mathcal{J}) = \mathrm{val}(\mathcal{J}_0)\, \tau\, \mathrm{val}(\mathcal{J}_1)\, \tau\, \mathrm{val}(\mathcal{J}_2) \cdots \tau\, \mathrm{val}(\mathcal{J}_{n-1})\, \tau^{-n+1}.$$

We can assume that $n \leq \ell + \sum \overline{r}_2 = \pi$ (n is a constant and we can always make ℓ bigger). Then $\mathrm{val}(\mathcal{J})$ evaluates to the group element $g \in G^{(\mathbb{Z})}$ with $g(x) = 1$ for $x \in \mathbb{Z} \setminus \{p_\beta - i \mid \beta \in \{0,1\}^{m_1}, 0 \leq i \leq n-1\}$ and $g(p_\beta - i) = g_i(p_\beta) = [f(p_\beta), a_i]$ for $0 \leq i \leq n-1$. Hence, if $f(p_\beta) \in Z(G)$ for all $\beta \in \{0,1\}^{m_1}$ then $\mathrm{val}(\mathcal{J}) = 1$ in $G \wr \mathbb{Z}$. On the other hand, if there is a $\beta \in \{0,1\}^{m_1}$ such that $f(p_\beta) \in G \setminus Z(G)$ then there is an a_i such that $[f(p_\beta), a_i] \neq 1$. Hence $g(p_\beta - i) \neq 1$ and $\mathrm{val}(\mathcal{J}) \neq 1$ in $G \wr \mathbb{Z}$. This proves the theorem. □

References

[1] Ian Agol. The virtual Haken conjecture. Documenta Mathematica, 18:1045–1087, 2013. With an appendix by Ian Agol, Daniel Groves, and Jason Manning.

[2] Sanjeev Arora and Boaz Barak. Computational Complexity - A Modern Approach. Cambridge University Press, 2009. doi:10.1017/CBO9780511804090.

[3] David A. Mix Barrington. Bounded-width polynomial-size branching programs recognize exactly those languages in NC^1 . Journal of Computer and System Sciences, 38(1):150–164, 1989. doi:10.1016/0022-0000(89)90037-8.

[4] David A. Mix Barrington and Denis Thérien. Finite monoids and the fine structure of NC^1 . Journal of the ACM, 35:941–952, 1988. doi:10.1145/48014.63138.

[5] Laurent Bartholdi, Michael Figelius, Markus Lohrey, and Armin Weiß. Groups with ALOGTIME-hard word problems and PSPACE-complete compressed word problems. Technical report, arXiv.org, 2020. arXiv:1909.13781.

[6] Laurent Bartholdi, Rostislav I. Grigorchuk, and Zoran Šuniḱ. Branch groups. In Handbook of algebra, Volume 3, pages 989–1112. Elsevier/North-Holland, Amsterdam, 2003. doi: 10.1016/S1570-7954(03)80078-5.

[7] Laurent Bartholdi and Volodymyr V. Nekrashevych. Iterated monodromy groups of quadratic polynomials. I. Groups, Geometry, and Dynamics, 2(3):309–336, 2008. doi:10.4171/GGD/42.

[8] Martin Beaudry, Pierre McKenzie, Pierre Péladeau, and Denis Thérien. Finite monoids: From word to circuit evaluation. SIAM Journal on Computing, 26(1):138–152, 1997. doi: 10.1137/S0097539793249530.

[9] Richard Beigel and John Gill. Counting classes: Thresholds, parity, mods, and fewness. Theoretical Computer Science, 103(1):3–23, 1992. doi:10.1016/0304-3975(92)90084-S.

[10] William W. Boone. The Word Problem. Annals of Mathematics, 70(2):207–265, 1959. URL: www.jstor.org/stable/1970103.

[11] Daniel P. Bovet, Pierluigi Crescenzi, and Riccardo Silvestri. A uniform approach to define complexity classes. Theoretical Computer Science, 104(2):263–283, 1992. doi:10.1016/ 0304-3975(92)90125-Y.

[12] John W. Cannon, William J. Floyd, and Walter R. Parry. Introductory notes on Richard Thompson's groups. L'Enseignement Mathématique, 42(3):215–256, 1996.

[13] Moses Charikar, Eric Lehman, Ding Liu, Rina Panigrahy, Manoj Prabhakaran, Amit Sahai, and Abhi Shelat. The smallest grammar problem. IEEE Transactions on Information Theory, 51(7):2554–2576, 2005. doi:10.1109/TIT.2005.850116.

[14] Max Dehn. Über unendliche diskontinuierliche Gruppen. Mathematische Annalen, 71(1):116– 144, 1911. doi:10.1007/BF01456932.

[15] Michael R. Garey and David S. Johnson. Computers and Intractability: A Guide to the Theory of NP–completeness. Freeman, 1979.

[16] Max Garzon and Yechezkel Zalcstein. The complexity of Grigorchuk groups with application to cryptography. Theoretical Computer Science, 88(1):83–98, 1991. doi:10.1016/0304-3975(91) 90074-C.

[17] Rostislav I. Grigorchuk. Burnside's problem on periodic groups. Functional Analysis and Its Applications, 14:41–43, 1980. doi:10.1007/BF01078416.

[18] Rostislav I. Grigorchuk and Zoran Šuniḱ. Asymptotic aspects of Schreier graphs and Hanoi Towers groups. C. R. Math. Acad. Sci. Paris, 342(8):545–550, 2006. doi:10.1016/j.crma. 2006.02.001.

[19] Victor S. Guba and Mark V. Sapir. On subgroups of the R. Thompson group F and other diagram groups. Matematicheskii Sbornik, 190(8):3–60, 1999. doi:10.1070/ SM1999v190n08ABEH000419.

[20] Narain Gupta and Saïd Sidki. On the Burnside problem for periodic groups. Mathematische Zeitschrift, 182(3):385–388, 1983. doi:10.1007/BF01179757.

[21] Frédéric Haglund and Daniel T. Wise. Coxeter groups are virtually special. Advances in Mathematics, 224(5):1890–1903, 2010. doi:10.1016/j.aim.2010.01.011.

[22] Ulrich Hertrampf. Über Komplexitätsklassen, die mit Hilfe von k-wertigen Funktionen definiert werden. Habilitationsschrift, Universität Würzburg, 1994.

[23] Ulrich Hertrampf. The shapes of trees. In Proceedings of COCOON 1997, volume 1276 of Lecture Notes in Computer Science, pages 412–421. Springer, 1997. doi:10.1007/BFb0045108.

[24] Ulrich Hertrampf. Algebraic acceptance mechanisms for polynomial time machines. SIGACT News, 31(2):22–33, 2000. doi:10.1145/348210.348215.

[25] Ulrich Hertrampf, Clemens Lautemann, Thomas Schwentick, Heribert Vollmer, and Klaus W. Wagner. On the power of polynomial time bit-reductions. In Proceedings of the Eighth Annual Structure in Complexity Theory Conference, pages 200–207. IEEE Computer Society Press, 1993. doi:10.1109/SCT.1993.336526.

[26] Ulrich Hertrampf, Heribert Vollmer, and Klaus Wagner. On balanced versus unbalanced computation trees. Mathematical Systems Theory, 29(4):411–421, 1996. doi:10.1007/BF01192696.

[27] Derek F. Holt, Markus Lohrey, and Saul Schleimer. Compressed decision problems in hyperbolic groups. In Proceedings of STACS 2019, volume 126 of LIPIcs, pages 37:1–37:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2019. URL: http://www.dagstuhl.de/dagpub/ 978-3-95977-100-9.

[28] Derek F. Holt, Sarah Rees, and Claas E. Röver. Groups, Languages and Automata, volume 88 of London Mathematical Society Student Texts. Cambridge University Press, 2017. doi: 10.1017/9781316588246.

[29] Birgit Jenner, Pierre McKenzie, and Denis Thérien. Logspace and logtime leaf languages. Information and Computation, 129(1):21–33, 1996. doi:10.1006/inco.1996.0071.

[30] Howard J. Karloff and Walter L. Ruzzo. The iterated mod problem. Information and Computation, 80(3):193–204, 1989. doi:10.1016/0890-5401(89)90008-4.

[31] Daniel König and Markus Lohrey. Evaluation of circuits over nilpotent and polycyclic groups. Algorithmica, 80(5):1459–1492, 2018. doi:10.1007/s00453-017-0343-z.

[32] Daniel König and Markus Lohrey. Parallel identity testing for skew circuits with big powers and applications. International Journal of Algebra and Computation, 28(6):979–1004, 2018. doi:10.1142/S0218196718500431.

[33] Jörg Lehnert and Pascal Schweitzer. The co-word problem for the Higman-Thompson group is context-free. Bulletin of the London Mathematical Society, 39(2):235–241, February 2007. doi:10.1112/blms/bdl043.

[34] Martin W. Liebeck, Eamonn A. O'Brien, Aner Shalev, and Pham Huu Tiep. The Ore conjecture. Journal of the European Mathematical Society, 12(4):939–1008, 2010. doi:10.4171/JEMS/220.

[35] Yury Lifshits and Markus Lohrey. Querying and embedding compressed texts. In Proceedings of MFCS 2006, volume 4162 of Lecture Notes in Computer Science, pages 681–692. Springer, 2006. doi:10.1007/11821069_59.

[36] Richard J. Lipton and Yechezkel Zalcstein. Word problems solvable in logspace. Journal of the Association for Computing Machinery, 24(3):522–526, 1977. doi:10.1145/322017.322031.

[37] Markus Lohrey. Leaf languages and string compression. Information and Computation, 209(6):951–965, 2011. doi:10.1016/j.ic.2011.01.009.

[38] Markus Lohrey. The Compressed Word Problem for Groups. Springer Briefs in Mathematics. Springer, 2014. doi:10.1007/978-1-4939-0748-9.

[39] Volodymyr Nekrashevych. Self-similar groups, volume 117 of Mathematical Surveys and Monographs. American Mathematical Society, Providence, RI, 2005. doi:10.1090/surv/117.

[40] Piotr S. Novikov. On the algorithmic unsolvability of the word problem in group theory. Trudy Mat. Inst. Steklov, pages 1–143, 1955. In Russian. URL: http://mi.mathnet.ru/eng/tm1180.

[41] David Robinson. Parallel Algorithms for Group Word Problems. PhD thesis, University of California, San Diego, 1993.

[42] Joseph J. Rotman. An Introduction to the Theory of Groups (fourth edition). Springer, 1995.

[43] Amir Shpilka and Amir Yehudayoff. Arithmetic circuits: A survey of recent results and open questions. Foundations and Trends in Theoretical Computer Science, 5(3-4):207–388, 2010. doi:10.1561/0400000039.

[44] Hans-Ulrich Simon. Word problems for groups and contextfree recognition. In Proceedings of FCT 1979, pages 417–422. Akademie-Verlag, 1979.

[45] Roman Smolensky. Algebraic methods in the theory of lower bounds for boolean circuit complexity. In Proceedings of STOC 1987, pages 77–82. ACM, 1987. doi:10.1145/28395. 28404.

[46] Jacques Tits. Free subgroups in linear groups. Journal of Algebra, 20(2):250–270, 1972. doi:10.1016/0021-8693(72)90058-0.

[47] Heribert Vollmer. Introduction to Circuit Complexity. Springer, Berlin, 1999. doi:10.1007/ 978-3-662-03927-4.

[48] Stephan Waack. The parallel complexity of some constructions in combinatorial group theory. Journal of Information Processing and Cybernetics EIK, 26:265–281, 1990. doi: 10.1007/BFb0029647.

[49] Jan Philipp Wächter and Armin Weiß. An automaton group with pspace-complete word problem. In Proceedings of STACS 2020, volume 154 of LIPIcs, pages 6:1–6:17. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020. URL: https://www.dagstuhl.de/dagpub/ 978-3-95977-140-5.

[50] John S. Wilson. Embedding theorems for residually finite groups. Mathematische Zeitschrift, 174(2):149–157, 1980. doi:10.1007/BF01293535.

[51] Daniel T. Wise. Research announcement: the structure of groups with a quasiconvex hierarchy. Electronic Research Announcements in Mathematical Sciences, 16:44–55, 2009. URL: http://aimsciences.org//article/id/8d3fc128-8d02-4fee-af67-38001ca1d0ac.